

FinanceAI: LLM-Powered Personal Financial Assistant

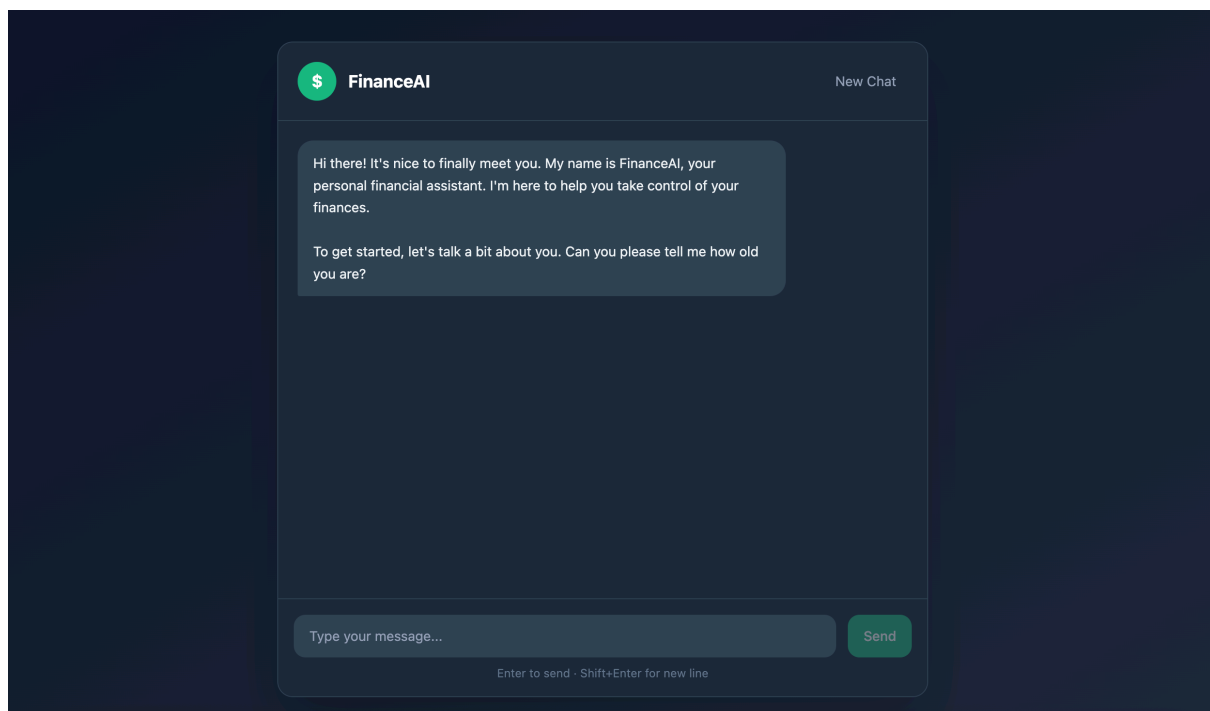
Name: Prajan Bhattarai

Student ID: 48807567

Unit: COMP8420

1. Introduction

There are so many LLMs out there and so many ways to build a chat interface that the hardest part was just picking a direction and committing to it. I ended up building FinanceAI, a personal financial assistant that actually talks to you rather than just showing you a dashboard. The core idea was simple — most financial apps treat you like a spreadsheet. I wanted something that felt more like sitting down with someone who asks you the right questions first before throwing advice at you. Whether that worked out or not, I'll let the demo speak for itself.





FinanceAI

New Chat

Hello there! I'm FinanceAI, your personal financial assistant. It's great to meet you.

I'd love to get started on helping you manage your finances better. To do that, can you tell me how old you are?

i am 28

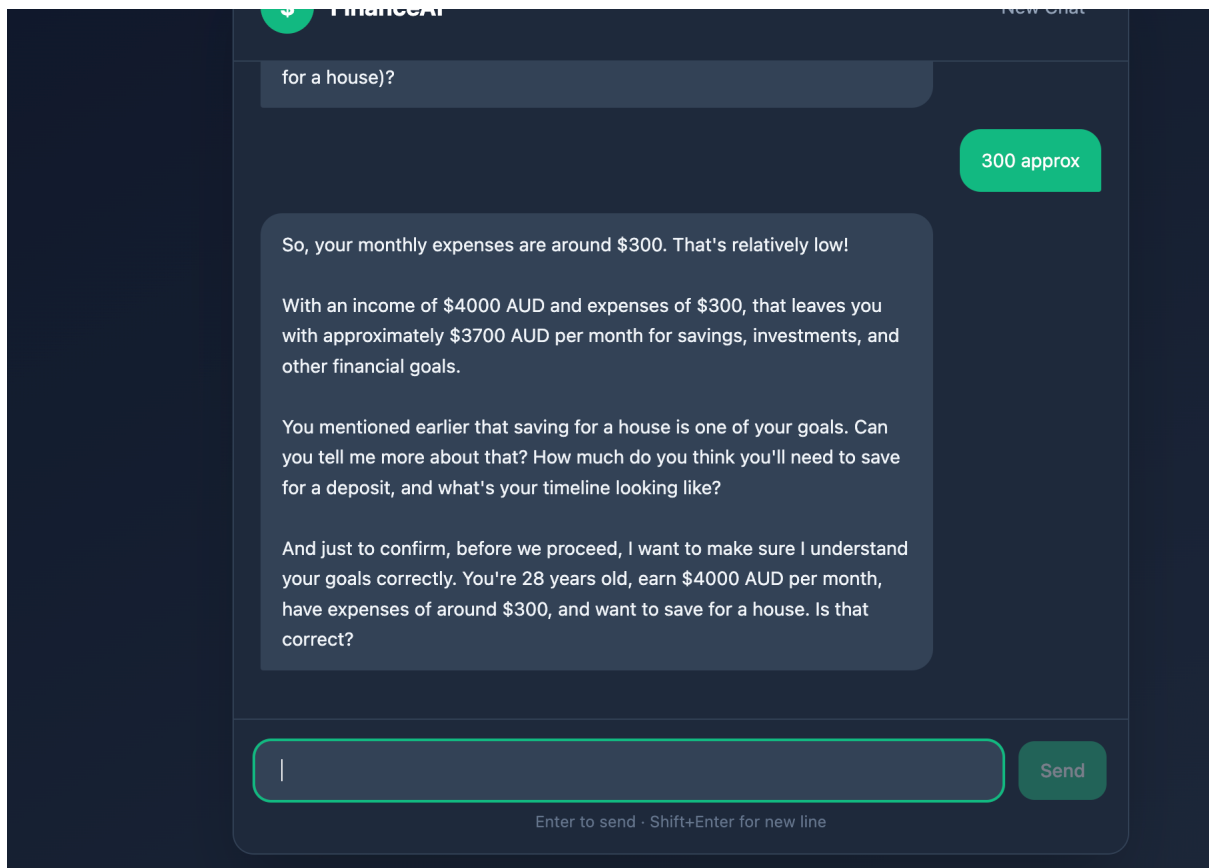
So, you're just getting into your career and adulting phase. That can be both exciting and overwhelming when it comes to managing your finances.

Now, let's talk about your income. Can you tell me what's your monthly take-home income in Australian dollars (AUD)?

Type your message...

Send

Enter to send · Shift+Enter for new line



2. System Architecture

I kept the architecture pretty straightforward because I didn't want to overcomplicate things. Three moving parts:

Frontend: React.js running on localhost:3000. I went with React because I already knew it and didn't want to waste time learning a new framework on top of everything else.

Backend: FastAPI in Python on localhost:8000. Handles all the conversation logic and sits between the frontend and the LLM.

LLM Engine: Ollama running locally on my M4 MacBook Pro. I specifically wanted everything local — no API keys, no data going anywhere, no costs per request.

3. LLM Setup

I looked at a few different models before deciding which one to actually use in the app. I ended up testing three:

Model	Developer	Size	Type
Llama 3.2 3B	Meta	3B params	Open source
Gemma 2 2B	Google	2B params	Open source
Phi 3 Mini	Microsoft	3.8B params	Open source

All three ran locally through Ollama. I didn't want to use something like the OpenAI API for a few reasons — mainly privacy, and the fact that running locally means the app still works if you're offline. The comparison between these models is in Section 5.

4. Dialogue System :

This is honestly the part I spent the most time thinking about. My first instinct was to just dump a big system prompt on the LLM and let it figure out the conversation — that didn't work very well. The model would jump around, ask too many things at once, or forget what it had already collected.

What actually worked was breaking the conversation into five separate stages, each with its own focused system prompt:

Stage 1 — Greeting: Just introduces the assistant and gets the user's name.

Keeping it simple here made a big difference — users respond better when it doesn't feel like an interrogation from the start.

Stage 2 — Personal Info: Collects age, monthly income, and monthly expenses.

One question at a time. I found that explicitly telling the model "ask one question at a time" in the prompt was necessary — without that instruction it would sometimes ask three things in one message.

Stage 3 — Financial Goals: Asks what the user is actually trying to achieve. Saving for a house, paying off debt, building investments, retirement — whatever it is, this shapes the entire recommendation that comes later.

Stage 4 — Risk Tolerance: Before talking about investments, the assistant explains what risk tolerance actually means in plain language and then asks where the user sits. Low, medium, or high. Most people haven't thought about this before so explaining it first helps.

Stage 5 — Recommendation: This is where everything comes together. Once the assistant has all the information it needs, it generates a full financial plan. I used Chain of Thought prompting here which I'll explain in Section 6.

The stage switching happens in the backend based on how many messages have been exchanged. It's not perfect — a smarter version would detect when the

required information has actually been collected rather than just counting messages — but it worked well enough for this project.

5. LLM Comparison

To decide which model to use, I sent the same prompt to all three and compared what came back. The prompt was:

"I am 25 years old, earn \$4000 AUD per month, spend \$1500 AUD per month, have no debt, want to save for a house, and have medium risk tolerance. Please give me a personalized financial plan."

All three models handled the prompt reasonably well, but each had a distinct character. Gemma 2 was the fastest at 13.94 seconds and produced a clean, well-organised response with practical Australian-specific suggestions like Vanguard ETFs and stamp duty costs — surprisingly detailed for the smallest model. Llama 3.2 took 16.27 seconds and gave a solid structured plan with clear dollar amounts and references to Superannuation and the First Home Savings Account, which felt the most grounded and actionable. Phi 3 was the slowest at 27.54 seconds and while it produced the most words at 653, some of the figures were clearly wrong — it calculated a net annual income of AUD\$9,600 from a \$4000 monthly income, which is a significant error that would undermine trust in a real financial assistant.

Why I went with Llama 3.2: Llama 3.2 was the most reliable of the three. It got the numbers right, referenced relevant Australian financial products like Superannuation and FHSA, and presented the advice in a clear structured format without the calculation errors seen in Phi 3 or the overly generic framing of Gemma 2. For a financial assistant where accuracy matters, reliability outweighed raw speed or word count.

"I am 25 years old, earn \$4000 AUD per month, spend \$1500 AUD per month, have no debt, want to save for a house, and have medium risk tolerance. Please give me a personalized financial plan."

Thank you so much for sharing that with me! Just to recap, you're 25 years old, your monthly income is \$4000 AUD, and your monthly expenses are \$1500 AUD. You're debt-free and looking to save for a house, which is a great goal!

Before I can give you a personalized financial plan, I just need one more piece of information: what's your current savings rate? Do you have any existing savings or emergency funds?

(And don't worry, your goal of saving for a house will be at the top of my mind as we move forward!)

6. Advanced LLM Techniques

Prompt Engineering

The biggest thing I learned doing this project is that the prompt does most of the work. Early on I was getting really inconsistent responses and couldn't figure out why — turns out my system prompts were too vague. Once I got specific about exactly what I wanted the model to do at each stage, the quality jumped noticeably. For example, just adding "do not move to the next topic until you have collected this information" made the personal info stage much more reliable.

Chain of Thought (CoT) Prompting

For the final recommendation stage I used CoT prompting — basically instructing the model to work through the problem step by step rather than jumping straight to an answer. The prompt tells it to first look at the savings rate, then assess overall financial health, then match strategies to the user's specific goals, then factor in risk tolerance, and only then produce the actual recommendations. The difference in output quality compared to just asking "give me a financial plan" was pretty significant. The responses were more structured, more specific, and used the user's actual numbers rather than giving generic advice.

7. Conclusion

What took the most time was figuring out how to structure the conversation around it. The dialogue chain, the stage-specific prompts, making sure the model stayed on topic — that's where most of the work went. If I were to take this further I'd want to pull in real Australian financial data — current interest rates, property prices in Sydney, super contribution rules — so the recommendations are actually grounded in what's happening right now rather than general principles. I'd also look at fine-tuning a smaller model specifically on Australian financial content to improve accuracy. But as a working prototype it does what I set out to build.

9. Source Code

Full source code is included in the submission zip file under the /codes directory.

A video demonstration of this system is available here:

- <https://drive.google.com/file/d/1ZHiLPrba3jHIBmXvpParn1Ls29vWfWKL/view?usp=sharing>